

## 10.2 lubridate: Dates and Times

Parse with `ymd()`, pull pieces out, and know duration vs period vs interval.

### 1. Load lubridate and look at the class

tidyverse attaches lubridate, but load it yourself so the parsers are obvious.

```
library(tidyverse)
library(lubridate)
```

Two classes matter. A **Date** is a calendar day. A **POSIXct** is a datetime: a Date plus a time of day, stored as seconds since 1970-01-01 UTC. `class()` tells you which you have.

```
ymd("2017-01-31")
ymd_hms("2017-01-31 20:11:59")
```

### 2. Parsers and constructors

Name the function after the order of the pieces. The separators can vary.

```
ymd("2017-01-31")
mdy("January 31st, 2017")
dmy("31-Jan-2017")
ymd_hms("2017-01-31 20:11:59")
mdy_hm("01/31/2017 08:15")
```

`make_date()` and `make_datetime()` build a date from numeric columns.

```
make_date(2017, 1, 31)
make_datetime(2017, 1, 31, 20, 11, 59)
```

Put the CSV files from the Data section of the course page in a `data/` folder of your project.

`wmata_ridership.csv` has a `Date` column (ISO text) and a `Total` column (daily Metro rail rides).

```
wmata <- read_csv("data/wmata_ridership.csv")
wmata <- wmata |>
  mutate(Date = ymd(Date))
```

### 3. Pull pieces out

```
dt <- ymd_hms("2016-07-08 12:34:56")
year(dt)
month(dt)
month(dt, label = TRUE)
day(dt)
wday(dt)
wday(dt, label = TRUE)
hour(dt)
```

`label = TRUE` on `month()` and `wday()` returns an ordered factor. `abbr = FALSE` spells the name out.

```
wmata |>
  mutate(
    year = year(Date),
    month = month(Date, label = TRUE),
    wday = wday(Date, label = TRUE)
  )
```

---

### 4. Durations, periods, and intervals

A **duration** is an exact number of seconds. `ddays(1)` is always 86,400 seconds.

A **period** is a human unit. `days(1)` is "one calendar day," which can be 23 or 25 hours when clocks change.

An **interval** is the span between two instants.

```
ddays(1)
days(1)
years(1)

ymd("2016-01-01") + ddays(365)
ymd("2016-01-01") + days(365)
ymd("2016-01-01") + years(1)

int <- interval(ymd("2004-01-01"), ymd("2018-12-21"))
int
as.duration(int)
```

2016 is a leap year, so `ddays(365)` and `days(365)` land on different days. Use a duration when you want elapsed time. Use a period when you want "same date next month." Use an interval when you have two endpoints.

---

## 5. Time zones

Datetimes default to **UTC**. `force_tz()` keeps the clock face and changes the zone (the instant moves). `with_tz()` keeps the instant and redraws the clock in another zone.

```
x <- ymd_hms("2020-03-08 01:30:00")
x
force_tz(x, "America/New_York")
with_tz(x, "America/New_York")
```

`OlsonNames()` lists the zone names. `"America/New_York"` is Eastern time, daylight saving included.

A clock that is missing a timezone is often stored as UTC by accident. On the spring-forward Sunday the computed duration can jump by an hour. `force_tz()` is the fix when the numbers on the clock were already Eastern.

```
toy <- tibble(
  start = mdy_hm(c("3/13/2016 1:30", "3/13/2016 1:30")),
  end   = mdy_hm(c("3/13/2016 3:30", "3/13/2016 3:30"))
)
toy |>
  mutate(
    start_ny = force_tz(start, "America/New_York"),
    end_ny   = force_tz(end, "America/New_York"),
    hours_utc = as.numeric(end - start) / 3600,
    hours_ny  = as.numeric(end_ny - start_ny) / 3600
  )
```

---

## 6. Practice

1. Parse `"2010-02-01"`, `"February 1, 2010"`, `"01-02-2010"`, and `"2010-02-01 08:15:00"`. `"01-02-2010"` is `dmy` or `mdy` depending on the country.
2. Build dates from this tribble and keep the rows before `2010-02-01`.

```
events <- tribble(
  ~year, ~month, ~day,
  2009,    12,   31,
  2010,     1,   15,
  2010,     2,    1,
  2010,     3,   10
)
```